Testing y Calidad de Código

Preguntas de reflexión y desarrollo de respuestas

13 preguntas

Índice

1. Pirámide de pruebas y estrategia de testing
2. Patrón AAA en pruebas complejas
3. Complejidad ciclomática y mantenibilidad
4. Comparación entre TDD y BDD
5. Relación entre Code Smells y Testing
6. Inyección de dependencias y testabilidad
7. Estrategias de mock: London School vs Classical TDD
8. Cobertura de código: métricas vs calidad de tests
9. Logging en arquitecturas distribuidas
10. Herramientas de análisis estático
11. Técnicas de caja negra: EP y BVA en metodologías ágiles
12. Documentación de tests y FluentAssertions
13. Test Doubles y el "teorema de Wiston"

1. Pirámide de pruebas y estrategia de testing

PREGUNTA

> *Analiza por qué la pirámide de pruebas (70% unitarias, 20% integración, 10% E2E) es una estrategia efectiva. ¿Qué problemas surgirían si invirtiéramos esta pirámide? Justifica tu respuesta con ejemplos de escenarios donde muchas pruebas E2E ralentizarían el desarrollo.*

Respuesta

La pirámide es eficaz porque equilibra **velocidad, fiabilidad y coste de diagnóstico**:

- **Unitarias:** se ejecutan en milisegundos, son deterministas y, cuando fallan, señalan la línea exacta.
- **Integración:** verifican contratos reales (BD, colas, HTTP) con un coste moderado.
- **E2E:** validan el sistema completo, pero son lentas, frágiles (*flaky*) y difíciles de depurar.

Problemas de invertir la pirámide (ice cream cone)

- **Feedback lento:** 500 tests E2E de 30 s suponen más de 4 horas en secuencial. El pipeline deja de ser un control rápido y los desarrolladores dejan de ejecutarlo.
- **Flakiness:** dependen de red, tiempos, datos y UI. Un fallo intermitente en CI genera desconfianza y la gente relanza el pipeline sin leer el error.
- **Diagnóstico caro:** si falla un E2E de "checkout", la causa puede estar en el front, la API de pagos, el stock o los datos de prueba. Un unitario falla y se sabe dónde.
- **Mantenimiento:** un cambio menor de UI (renombrar un selector) rompe decenas de tests.
- **Entornos:** necesitan un entorno completo y datos consistentes, lo que genera colas y conflictos entre equipos.

Ejemplo

Un equipo cambia una regla de descuento. Con tests unitarios ejecuta 20 casos en 1 s. Con solo E2E tiene que levantar el entorno, recorrer el flujo de compra con cada combinación de cliente y esperar unos 40 minutos para saber si rompió algo.

> *Matiz: 70/20/10 es una heurística, no una ley. En aplicaciones con mucha lógica de integración (CRUD sobre BD) muchos equipos prefieren un "trofeo de testing" con más peso en integración.*

2. Patrón AAA en pruebas complejas

PREGUNTA

> *En pruebas de servicios que dependen de múltiples repositorios y servicios externos, ¿cómo se aplica el patrón AAA? Investiga cómo el "Arrange" puede volverse complejo y qué estrategias existen para mantener los tests legibles y mantenibles.*

Respuesta

Arrange-Act-Assert se mantiene, pero el *Arrange* crece mucho cuando el servicio depende de varios repositorios y servicios externos: múltiples mocks, `Setup` encadenados y entidades con muchos campos obligatorios.

Estrategias para mantenerlo legible

- **Test Data Builders / Object Mother:** `new OrderBuilder().WithCustomer(vip).WithItems(3).Build()`. Solo se especifica lo relevante para el caso.
- **Factorías de fixtures** o métodos privados con nombres de intención (`GivenCustomerWithoutCredit()`).
- **AutoFixture + AutoMoq/NSubstitute:** crea el SUT y sus dependencias automáticamente.
- **Constructor/SetUp común** solo para lo verdaderamente compartido, evitando el "misterio del invitado" (que el test dependa de datos ocultos en otro lugar).
- **Fakes en memoria** en lugar de decenas de `Setup`: un `InMemoryOrderRepository` reutilizable es más legible que un mock configurado en cada test.
- **Un Act por test** y asserts sobre un único comportamiento.

> *Señal de alarma: si el Arrange ocupa 40 líneas, probablemente el servicio viola SRP y tiene demasiadas dependencias. El test está avisando de un problema de diseño.*

3. Complejidad ciclomática y mantenibilidad

PREGUNTA

> *La complejidad ciclomática de McCabe mide el número de rutas independientes en un código. Investiga cómo un valor alto de complejidad (por ejemplo, > 10) afecta a la capacidad de probar y mantener el código. ¿Cómo se relaciona esto con los principios SOLID?*

Respuesta

La complejidad ciclomática (CC) es el número de caminos linealmente independientes (aproximadamente decisiones + 1). Con CC > 10:

- **Testabilidad:** se necesitan al menos CC casos para cubrir todos los caminos. Con CC = 15 son 15 o más tests solo para esa función, con combinatoria de condiciones entre ramas.
- **Comprensión:** supera la capacidad de la memoria de trabajo; el desarrollador no puede razonar sobre todas las rutas.
- **Defectos:** la evidencia empírica muestra una correlación moderada con la densidad de bugs y el coste de cambio.
- **Refactorización arriesgada:** cualquier cambio puede afectar a una ruta no cubierta.

Relación con SOLID

- **SRP:** una CC alta suele indicar que un método hace varias cosas.
- **OCP:** las cadenas de `if/switch` por tipo se sustituyen por polimorfismo o Strategy, lo que baja la CC y permite extender sin modificar.
- **LSP/DIP:** al delegar decisiones a abstracciones inyectadas, cada clase tiene menos ramas y se prueba de forma aislada.

> *Matiz: la CC es un indicador, no un veredicto. Un `switch` plano con 12 casos triviales tiene CC alta pero es fácil de leer. La complejidad cognitiva (SonarQube) corrige esta limitación penalizando el anidamiento.*

4. Comparación entre TDD y BDD

PREGUNTA

> *Investiga y compara TDD (Test Driven Development) y BDD (Behavior Driven Development). ¿En qué tipo de proyectos es más apropiado cada enfoque? ¿Cómo afectan a la comunicación entre el equipo técnico y los stakeholders?*

Respuesta

	TDD	BDD
Foco	Diseño del código (Red-Green-Refactor)	Comportamiento visible para el negocio
Lenguaje	Código (xUnit)	Gherkin: Given/When/Then (SpecFlow/Reqnroll, Cucumber)
Audiencia	Desarrolladores	Desarrolladores, QA, producto y stakeholders
Nivel típico	Unitario	Aceptación / integración

Cuándo usar cada uno

- **TDD:** lógica de dominio compleja, librerías, algoritmos, APIs internas, equipos técnicos.
- **BDD:** productos con reglas de negocio ambiguas o cambiantes, múltiples stakeholders y dominios regulados (banca, seguros), donde el malentendido de requisitos es el riesgo principal.

Impacto en la comunicación

BDD crea un lenguaje ubicuo y los escenarios sirven como criterios de aceptación ejecutables y documentación viva (sesiones "Three Amigos"). TDD mejora la comunicación dentro del equipo técnico, pero los stakeholders no leen los tests.

Riesgo de BDD: si el negocio no participa, los ficheros .feature se convierten en tests verbosos de UI sin valor añadido. Son complementarios: BDD como bucle externo y TDD como bucle interno.

5. Relación entre Code Smells y Testing

PREGUNTA

> *Los code smells indican problemas en el código que pueden llevar a bugs. Investiga cómo los principios de Clean Code y las técnicas de refactorización se relacionan con la capacidad de escribir tests efectivos. ¿Puede un código con muchos code smells ser correctamente testeado?*

Respuesta

Clean Code y la refactorización son prerrequisitos de la testabilidad: funciones pequeñas con una responsabilidad, pocas dependencias y bajo acoplamiento son fáciles de aislar. Smells como *God Class*, *Long Method*, *Feature Envy*, estado global, singletons y dependencias ocultas dificultan el aislamiento.

¿Se puede testear código con muchos smells?

Sí, pero con un coste alto. La respuesta práctica (Michael Feathers, *Working Effectively with Legacy Code*) es:

1. Escribir **characterization tests** que fijen el comportamiento actual, aunque sea a alto nivel.
2. Introducir **seams** (puntos de sustitución) con refactors mínimos y seguros.
3. Refactorizar bajo la red de seguridad de esos tests.
4. Escribir tests más finos sobre el código ya limpio.

Los tests habilitan la refactorización y esta mejora la testabilidad: un ciclo virtuoso. Los tests de código con smells tienden a ser frágiles, lentos y sobreacoplados a la implementación.

6. Inyección de dependencias y testabilidad

PREGUNTA

> *Investiga cómo la inyección de dependencias (DI) facilita el uso de mocks y stubs. ¿Por qué es difícil testear código que crea sus propias dependencias con `new`? Analiza el patrón Service Locator como alternativa a DI.*

Respuesta

Por qué `new` dentro de una clase dificulta el test

```
public class OrderService {
    private readonly SqlRepo _repo = new SqlRepo("connString"); // acoplamiento fijo
    private readonly SmtpClient _mail = new SmtpClient();
}
```

No hay forma de sustituir `SqlRepo` o `SmtpClient`: el test toca BD y correo reales, es lento, no determinista y tiene efectos secundarios. Es acoplamiento a implementaciones concretas (viola DIP).

Con DI (por constructor)

```
public OrderService(IOrderRepository repo, IEmailSender mail) { ... }
```

El test inyecta un **stub** (devuelve datos fijos), un **mock** (verifica interacciones) o un **fake** (implementación en memoria). Además, las dependencias quedan explícitas en la firma.

Service Locator (`ServiceLocator.Get<IRepo>()`)

- **A favor:** desacopla de la implementación concreta y permite sustituir en tests.
- **En contra:** oculta las dependencias (no se ven en el constructor), crea acoplamiento al locator, falla en runtime en vez de compilar y hace los tests más frágiles (hay que configurar el locator global y limpiarlo entre tests).

Mark Seemann y Martin Fowler lo consideran un anti-patrón en la mayoría de los casos. Es tolerable en el *composition root*, en código legacy o en frameworks donde DI no es posible.

7. Estrategias de mock: London School vs Classical TDD

PREGUNTA

> *Existen dos escuelas principales de TDD: la London School (mockist) que usa mocks para todo, y la Classical School que prefiere objetos reales cuando es posible. Investiga las ventajas y desventajas de cada enfoque y en qué escenarios es preferible cada uno.*

Respuesta

London School (mockist)

Se aísla cada clase sustituyendo sus colaboradores por mocks. Se diseña "de fuera hacia dentro" (*outside-in*), verificando interacciones. Nota: no se mockea "todo", sino los colaboradores con comportamiento relevante; los value objects suelen usarse reales.

- **Ventajas:** aislamiento total, diagnóstico preciso, ayuda a descubrir interfaces y responsabilidades, útil cuando los colaboradores aún no existen.
- **Desventajas:** tests acoplados a la implementación (se rompen al refactorizar aunque el comportamiento sea correcto), mucho setup, riesgo de que los mocks diverjan de la realidad.

Classical (Chicago/Detroit)

Usa objetos reales y reserva los dobles para dependencias *out-of-process* (BD, red, reloj). Verifica estado o resultado.

- **Ventajas:** tests más robustos ante refactors, más cercanos al comportamiento real, menos código de test.
- **Desventajas:** un fallo puede originarse en varias clases (diagnóstico menos preciso) y los grafos de objetos grandes complican el Arrange.

Cuándo usar cada uno

London para arquitecturas con muchos colaboradores y orquestación (servicios de aplicación, handlers, diseño outside-in). Classical para lógica de dominio, cálculos y algoritmos. En la práctica muchos equipos combinan ambos: mocks solo en los límites del sistema (idea defendida por Vladimir Khorikov).

8. Cobertura de código: métricas vs calidad de tests

PREGUNTA

> *La cobertura de código mide qué porcentaje del código es ejecutado por tests, pero un 100% de cobertura no garantiza tests de calidad. Investiga qué otras métricas y prácticas complementan la cobertura para garantizar tests efectivos. ¿Cómo evitar el "test pollution"?*

Respuesta

Un test sin asserts puede dar 100% de cobertura sin verificar nada. Complementos:

- **Mutation testing** (Stryker.NET): introduce mutaciones en el código y comprueba si los tests las detectan. El *mutation score* mide la capacidad real de detectar fallos.
- **Cobertura de ramas/condiciones**, no solo de líneas.
- **Revisión de asserts:** ¿verifican comportamiento relevante?
- **Tasa de defectos escapados** y tiempo de detección de regresiones.
- **Tasa de flakiness** y tiempo de ejecución del suite.
- **Cobertura sobre código nuevo** (*new code coverage* en SonarQube) en lugar de un porcentaje global.
- **Property-based testing** (FsCheck) y revisión de código de los tests.

Evitar el "test pollution"

Se refiere a tests que interfieren entre sí o ensucian el suite:

- Aislar el estado: no usar estáticos/singletons compartidos; cada test crea su propio contexto.
- No depender del orden de ejecución; que sean ejecutables en paralelo.
- Limpiar recursos (`IDisposable`, transacciones con rollback, contenedores efímeros con Testcontainers).
- Evitar lógica condicional en los tests y asserts sobre detalles de implementación.
- Eliminar tests duplicados u obsoletos y tratar el código de test con el mismo cuidado que el de producción.

9. Logging en arquitecturas distribuidas

PREGUNTA

> *En microservicios o aplicaciones distribuidas, el logging se vuelve crítico. Investiga cómo usar correlación de IDs (correlation IDs) para rastrear requests a través de múltiples servicios. ¿Cómo afecta esto a la configuración de Serilog?*

Respuesta

Un **correlation ID** es un identificador único generado en el punto de entrada (gateway o primer servicio) y propagado en cada llamada, de modo que todos los logs de una petición pueden filtrarse juntos.

Cómo funciona

5. Si llega el header (`X-Correlation-ID`) se reutiliza; si no, se genera uno.
6. Se propaga en las llamadas salientes (un `DelegatingHandler` en `HttpClient`) y en los mensajes (headers de la cola).
7. Se incluye en cada entrada de log.

Alternativa estándar: **W3C Trace Context** (`traceparent`) con OpenTelemetry. En .NET, `Activity.Current.TraceId` ya lo gestiona.

Configuración con Serilog

```
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()   // imprescindible
    .WriteTo.Console(outputTemplate:
        "{Timestamp:HH:mm:ss} [{Level}] ({CorrelationId}) {Message}{NewLine}{Exception}")
    .CreateLogger();

// Middleware
app.Use(async (ctx, next) => {
    var id = ctx.Request.Headers["X-Correlation-ID"].FirstOrDefault()
             ?? Guid.NewGuid().ToString();
    ctx.Response.Headers["X-Correlation-ID"] = id;
    using (LogContext.PushProperty("CorrelationId", id))
        await next();
});
```

Existe también el paquete `Serilog.Enrichers.CorrelationId`. Conviene usar un sink estructurado (Seq, Elasticsearch, Application Insights) para consultar por `CorrelationId`.

> *Cuidado con hilos/async: `LogContext` usa `AsyncLocal`, por lo que fluye con `await`, pero no en procesos fuera del contexto (por ejemplo, jobs en segundo plano, que hay que inicializar explícitamente).*

10. Herramientas de análisis estático

PREGUNTA

> *Investiga cómo SonarLint, SonarQube y StyleCop se complementan en un flujo de trabajo de desarrollo. ¿En qué se diferencian? ¿Puede una herramienta reemplazar a otra? ¿Cómo se integran en CI/CD?*

Respuesta

Herramienta	Dónde corre	Qué hace
StyleCop (Analyzers)	Compilación (Roslyn)	Estilo y convenciones: orden de usings, espacios, documentación XML, nombres
SonarLint	IDE (VS / Rider / VS Code)	Feedback en tiempo real: bugs, vulnerabilidades, code smells; puede enlazarse con el servidor (*connected mode*)
SonarQube	Servidor	Análisis global, histórico, deuda técnica, cobertura, duplicación, hotspots de seguridad y Quality Gates

Diferencias

StyleCop se centra en **formato y estilo**, SonarLint en **calidad y seguridad mientras se escribe**, y SonarQube en **gobierno de calidad a nivel de proyecto/equipo**.

¿Puede una reemplazar a otra?

No del todo. Sonar cubre poco el estilo fino, StyleCop no detecta bugs ni vulnerabilidades, y SonarLint no ofrece métricas históricas ni quality gates. Lo ideal es combinarlas: StyleCop y `.editorconfig` para el estilo, SonarLint en el IDE y SonarQube en CI.

Integración en CI/CD

- Secuencia típica: `dotnet sonarscanner begin` → `dotnet build` → `dotnet test` (con cobertura) → `dotnet sonarscanner end`.
- StyleCop se aplica en el build (con `TreatWarningsAsErrors` o reglas en `.editorconfig`).
- El Quality Gate bloquea el merge en la PR (*PR decoration*).

11. Técnicas de caja negra: EP y BVA en metodologías ágiles

PREGUNTA

> *El Particionamiento de Equivalencia (EP) y el Análisis de Valores Límite (BVA) son técnicas para diseñar casos de prueba desde los requisitos. Investiga cómo estas técnicas se aplican en metodologías ágiles donde los requisitos evolucionan constantemente.*

Respuesta

- **EP:** divide las entradas en clases que el sistema trata igual y prueba un representante de cada una.
- **BVA:** prueba los valores en y alrededor de los límites, donde se concentran los errores (off-by-one).

Aplicación en ágil con requisitos cambiantes

- Se derivan de los **criterios de aceptación** de cada historia de usuario, en el refinamiento (junto a los *Three Amigos*).
- Se implementan como **tests parametrizados** (`[Theory]`/`[InlineData]`, `[TestCase]`): si cambia el límite (por ejemplo, edad mínima de 18 a 16), se modifican los datos y no la estructura del test.
- Las particiones se mantienen **ligeras** y se revisan por iteración: solo se afinan en las historias que se van a implementar (*just-in-time*).
- Los límites se definen en **constantes o configuración**, para que tests y código cambien en un único lugar.
- Se complementan con **testing exploratorio** y **property-based testing** para cubrir lo que las particiones no anticipan.
- Se integran en la suite de regresión automatizada, de forma que un cambio de requisito rompe de forma visible los tests afectados.

12. Documentación de tests y FluentAssertions

PREGUNTA

> *Los tests deben ser documentación viviente del código. Investiga qué prácticas hacen que los tests sean auto-documentados. ¿Cómo afecta el uso de FluentAssertions a la legibilidad de los tests comparado con Assert tradicionales?*

Respuesta

Prácticas de tests auto-documentados

- Nombres que expresan escenario y resultado (`Withdraw_WhenBalanceIsInsufficient_ThrowsInsufficientFundsException`).
- Estructura AAA clara, con un solo comportamiento por test.
- Datos de prueba significativos (builders con nombres de intención, no `"abc"` o `123`).
- Sin lógica (bucles, condicionales) en el test.
- Mensajes de fallo descriptivos y organización por funcionalidad.
- Escenarios Gherkin si hay stakeholders implicados.

FluentAssertions vs Assert tradicional

```
// Tradicional
Assert.Equal(3, result.Items.Count);
Assert.True(result.Total > 0);

// Fluent
result.Items.Should().HaveCount(3);
result.Total.Should().BeGreaterThan(0);
result.Should().BeEquivalentTo(expected, o => o.Excluding(x => x.Id));
```

- **Ventajas:** lectura casi en lenguaje natural, mensajes de error mucho más informativos, comparación estructural (`BeEquivalentTo`), aserciones sobre colecciones, excepciones y fechas.
- **Inconvenientes:** curva de aprendizaje y riesgo de cadenas excesivamente largas.

> *Nota práctica: desde la versión 8, FluentAssertions cambió a una licencia comercial para uso empresarial, por lo que conviene revisarlo. Alternativas: Shouldly y AwesomeAssertions (fork con licencia abierta).*

13. Test Doubles y el "teorema de Wiston"

PREGUNTA

> *El teorema de Wiston establece que "mocks revelan el diseño". Investiga qué significa esto en la práctica. ¿Cómo el uso de mocks puede mejorar o empeorar el diseño de nuestro código?*

Respuesta

> *Aclaración importante: no se conoce ningún "teorema de Wiston" en la literatura. La idea de que los mocks revelan el diseño procede de la comunidad London School / Mockist, en particular de Steve Freeman y Nat Pryce (*Growing Object-Oriented Software, Guided by Tests*; artículo "Mock Roles, Not Objects"). Conviene verificar la fuente del material, ya que puede tratarse de un error de nombre o de una atribución del curso; no debería citarse como "teorema" sin comprobarlo.*

Qué significa en la práctica

Si un test requiere muchos mocks, configuraciones complejas o mockear cosas que no deberían poder mockearse, el código está avisando de un mal diseño:

- **Muchos mocks** → demasiadas dependencias → violación de SRP.
- **Mocks de mocks** (cadenas `a.B.C.D`) → violación de la Ley de Demeter.
- **Dificultad para mockear** una dependencia → acoplamiento a clases concretas o estáticas.
- **Interfaces que emergen** al escribir el test → roles y contratos más claros (*tell, don't ask*).

Cómo pueden mejorar el diseño

Guían hacia interfaces pequeñas y orientadas a roles, bajo acoplamiento y objetos con comportamiento.

Cómo pueden empeorarlo

El mocking excesivo acopla los tests a detalles internos (verificar el orden exacto de llamadas), fomenta interfaces 1:1 con cada clase solo "para poder mockear" y da falsa confianza si el mock no refleja la dependencia real.

Regla práctica: **mockear roles y límites del sistema, no detalles internos**, y tratar el dolor del test como señal para rediseñar.